

Overview

[Github refactor branch link](#)

This iteration of the class project implements DOT graph parsing with support for multiple search algorithms, including **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Random Walk Search**.

The project focuses on improving the code quality and readability through refactoring and applying software design patterns according to the requirements of the project description:

- **Template Method Pattern**
 - **Strategy Pattern**
-

How to Build and Run the Project

- **Build the Project in IntelliJ terminal**

Run: mvn package

- **Example test code (place either inside main or test)**

```
Graph graph = new Graph();  
graph.parseGraph("input.dot");
```

```
MutableNode src = graph.findNode("a");  
MutableNode dst = graph.findNode("c");
```

```
graph.GraphSearch(src, dst, Graph.Algorithm.BFS);  
graph.GraphSearch(src, dst, Graph.Algorithm.DFS);  
graph.GraphSearch(src, dst, Graph.Algorithm.RANDOM);
```

Template Method Pattern

The **Template Method Pattern** was applied to extract the common logic between BFS and DFS into a shared abstract class.

Implementation

A base class **GraphSearchTemplate** defines the overall search algorithm:

- Initialize each of the data structures
- Iterate through the nodes
- Check the goal condition
- Process node neighbors
- Build the final path to return

```
public Graph.Path search(MutableNode src, MutableNode dst)
```

This method defines the **fixed structure** of the algorithm, while allowing each of the new subclasses to customize behavior.

Abstract Methods

The following methods are implemented differently by each algorithm:

- **init(src)** → initializes traversal structure
 - **isEmpty()** → determines termination
 - **getNext()** → retrieves next node
 - **addNode(node)** → adds node to structure
-

Benefit

This pattern:

- Eliminates duplicated logic between BFS and DFS
 - Centralizes algorithm flow
 - Makes it easy to add new search algorithms
-

Strategy Pattern

The **Strategy Pattern** was applied to allow dynamic selection of the search algorithm at runtime.

Implementation

A common interface was added through the addition of SearchStrategy.java:

```
public interface SearchStrategy {  
    Graph.Path search(MutableNode src, MutableNode dst);  
}
```

Each supported algorithm implements this interface:

- BFS
 - DFS
 - RandomWalk
-

Algorithm Selection

The **GraphSearch** method within Graph.java selects the strategy based on the **Algorithm** enum:

```
switch (algo) {  
    case BFS:  
        strategy = new BFS(this);  
        break;  
    case DFS:  
        strategy = new DFS(this);  
        break;  
    case RANDOM:  
        strategy = new RandomWalk(this);  
        break;  
}
```

Benefit

This pattern:

- Makes the system extensible for future refactoring
 - Allows new algorithms to be added without modifying existing logic
-

Random Walk Algorithm

A new search algorithm, **Random Walk**, was implemented.

Example Behavior

Visit Node History: a

Visit Node History: a-e

Visit Node History: a-e-g

Visit Node History: a-e-g-h

Reached dead end at node: h

Randomness Requirement

The algorithm was executed multiple times during the demo to demonstrate its randomness:

```
for (int i = 0; i < 3; i++) {  
    graph.GraphSearch(a, c, Algorithm.RANDOM);  
}
```

Each run of the algorithm has the potential to produce a different path.

Random Walk extends the **GraphSearchTemplate** (Template Pattern) while also implementing **SearchStrategy** (Strategy Pattern).

Refactoring Summary

In total five refactorings were performed on the codebase:

1. Added VisitHistoryDisplay functionality:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/e5361731f9a0cb86de86ff92d77fa944db9776bc>
2. Extracted shared BFS/DFS logic into **GraphSearchTemplate** :
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/0f40f39dc3d5ab6a9016ae7ec6b1f3f9af0867a7>
3. Added a new **SearchStrategy** interface:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/0f40f39dc3d5ab6a9016ae7ec6b1f3f9af0867a7>
4. Separated BFS, DFS, and RandomWalk into their respective classes:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/a25ae6c2c00d270719bbd83d27e9f44aba55e4fc>
5. Removed duplicated traversal logic from **Graph** :
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/1bda7c18fcbfb06560006e0f597c9b4f80136c2c>

Example Code Output

BFS

Visit Node History: a
Visit Node History: a-b
Visit Node History: a-c
Found target node: c

DFS

Visit Node History: a
Visit Node History: a-b
Visit Node History: a-b-c
Found target node: c

Random Walk

Visit Node History: a
Visit Node History: a-e
Visit Node History: a-e-f
Visit Node History: a-e-f-c
Found target node: c